Kernix 0.10.0 / Performance brief

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · µs · NumPy BLAS: scipy-openblas

31 workloads

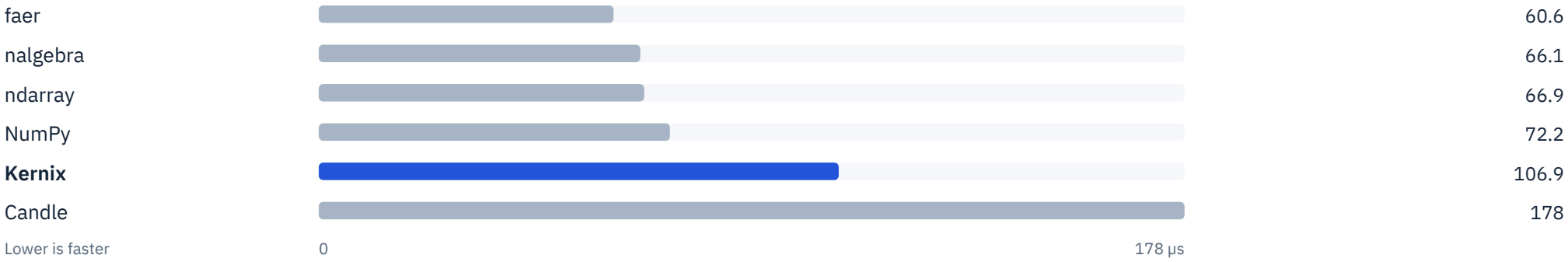
108 observations

Source precision retained

Matrix multiplication

Per-call median

128 × 128 · 128 × 128 · f64 · 1 worker · allocated output · self product



How to read this report

Allocation-inclusive API latency unless marked reused output. Peer medians are observations.

Charts round to 6 significant digits (≈). The exact-value appendix preserves all recorded digits.

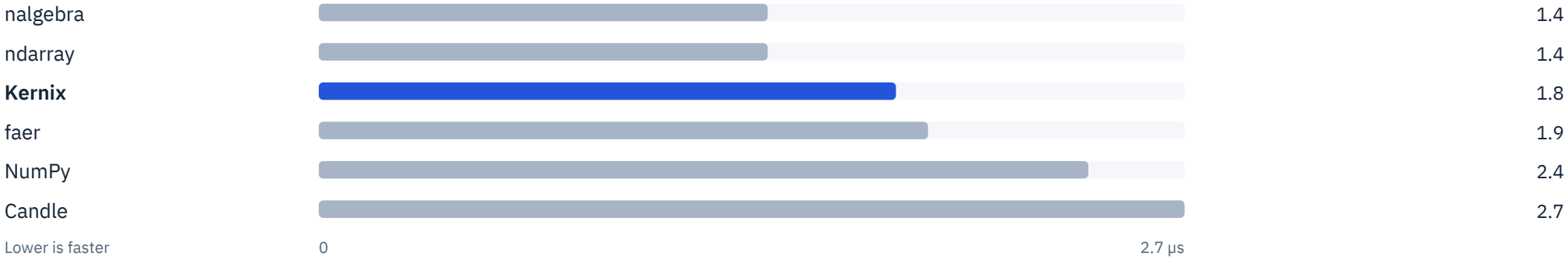
Kernix 0.10.0 / Matrix multiplication

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · µs · NumPy BLAS: scipy-openblas

Matrix multiplication

Per-call median

32 × 32 · 32 × 32 · f64 · 1 worker · allocated output · self product



Matrix multiplication

Per-call median

64 × 64 · 64 × 64 · f64 · 1 worker · allocated output · self product



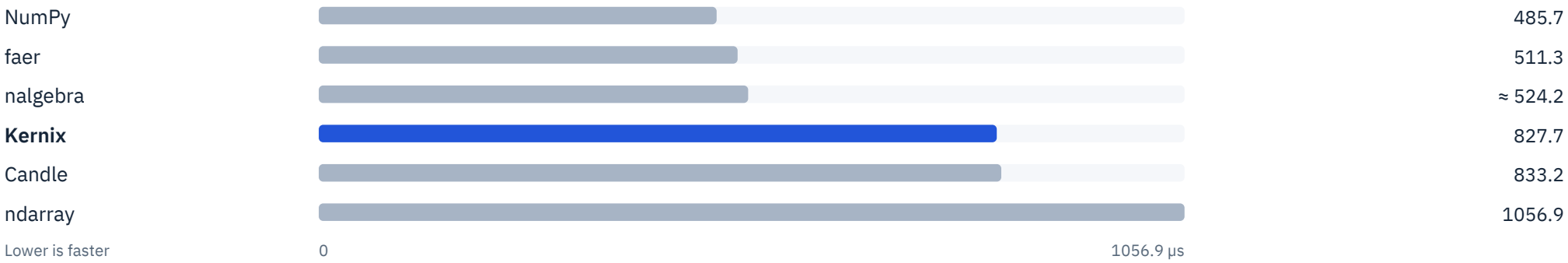
Kernix 0.10.0 / Matrix multiplication

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · µs · NumPy BLAS: scipy-openblas

Matrix multiplication

Per-call median

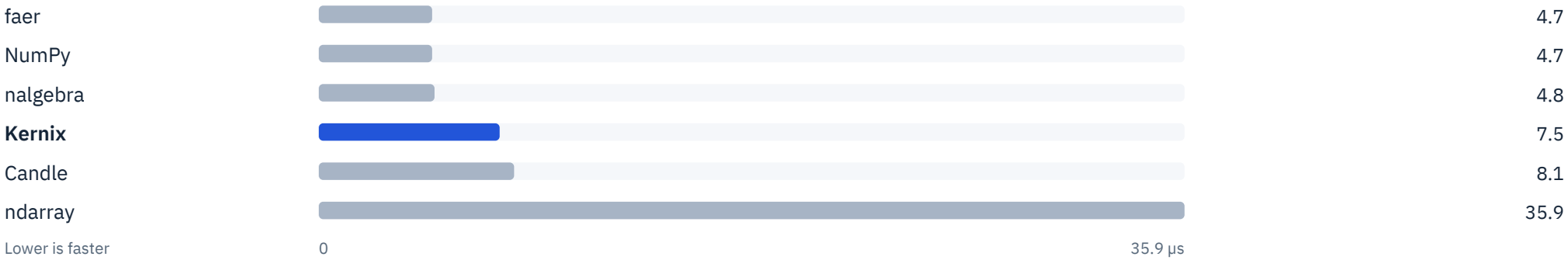
256 × 256 · 256 × 256 · f64 · 1 worker · allocated output · self product



Matrix multiplication

Per-call median

64 × 64 · 64 × 64 · f32 · 1 worker · allocated output · self product



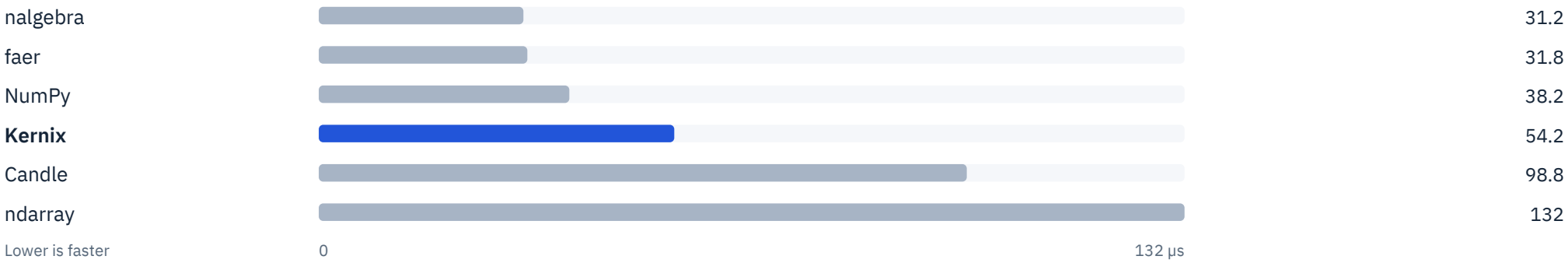
Kernix 0.10.0 / Matrix multiplication

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · µs · NumPy BLAS: scipy-openblas

Matrix multiplication

Per-call median

128 × 128 · 128 × 128 · f32 · 1 worker · allocated output · self product



Matrix multiplication

Per-call median

256 × 256 · 256 × 256 · f32 · 1 worker · allocated output · self product

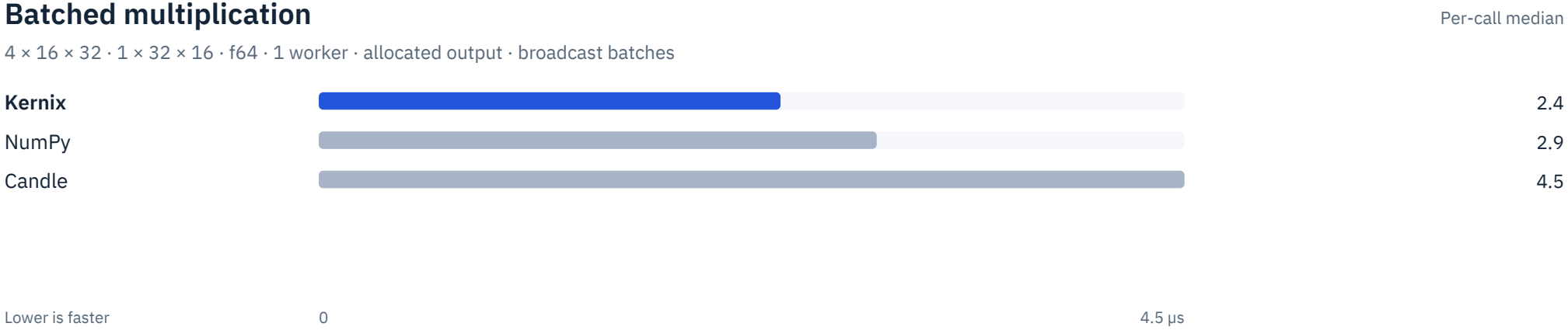


Kernix 0.10.0 / Matrix multiplication

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · µs · NumPy BLAS: scipy-openblas

Batched multiplication

4 × 16 × 32 · 1 × 32 × 16 · f64 · 1 worker · allocated output · broadcast batches



Kernix 0.10.0 / Explicit parallel execution

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · µs · NumPy BLAS: scipy-openblas

Matmul · 2 workers · reused output

Per-call median

64 × 64 · 64 × 64 · f64 · 2 workers · reused output · pool preparation excluded



Matmul · 2 workers · reused output

Per-call median

128 × 128 · 128 × 128 · f64 · 2 workers · reused output · pool preparation excluded



Kernix 0.10.0 / Explicit parallel execution

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · µs · NumPy BLAS: scipy-openblas

Matmul · 2 workers · reused output

Per-call median

256 × 256 · 256 × 256 · f64 · 2 workers · reused output · pool preparation excluded



Matmul · 2 workers · reused output

Per-call median

128 × 128 · 128 × 128 · f32 · 2 workers · reused output · pool preparation excluded



Kernix 0.10.0 / Decompositions & solvers

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · µs · NumPy BLAS: scipy-openblas

Column-pivoted QR

96 × 32 · f64 · 1 worker · allocated output

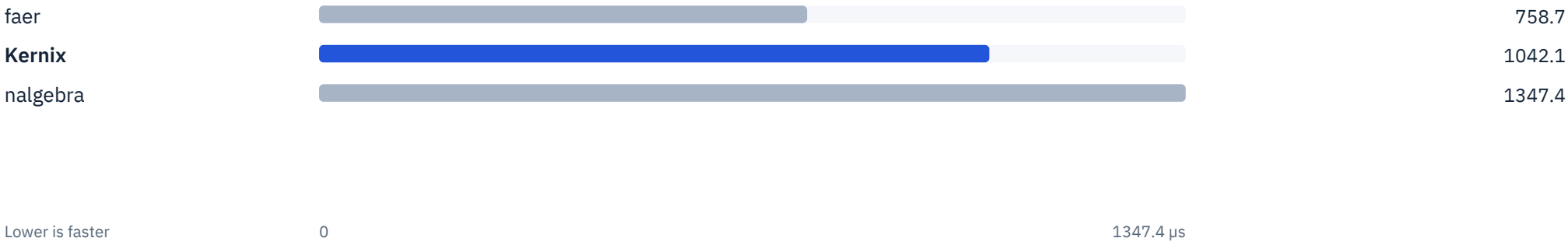
Per-call median



Column-pivoted QR

256 × 64 · f64 · 1 worker · allocated output

Per-call median



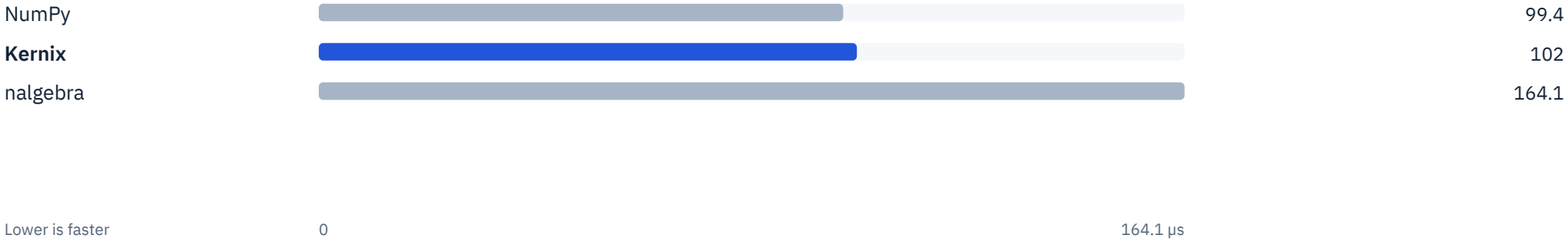
Kernix 0.10.0 / Decompositions & solvers

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · µs · NumPy BLAS: scipy-openblas

Minimum-norm least squares

Per-call median

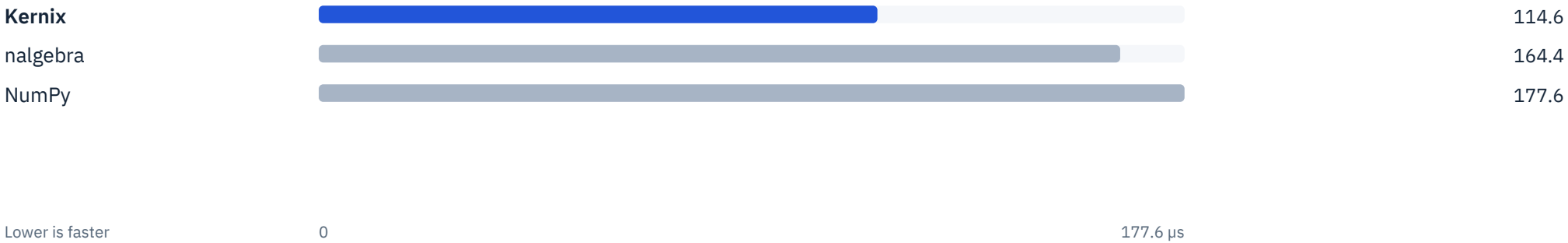
64 × 32, rank 16, 4 RHS · f64 · 1 worker · 4 RHS · factorization included · algorithms differ



Minimum-norm least squares

Per-call median

32 × 64, rank 32, 4 RHS · f64 · 1 worker · 4 RHS · factorization included · algorithms differ



Kernix 0.10.0 / Decompositions & solvers

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · µs · NumPy BLAS: scipy-openblas

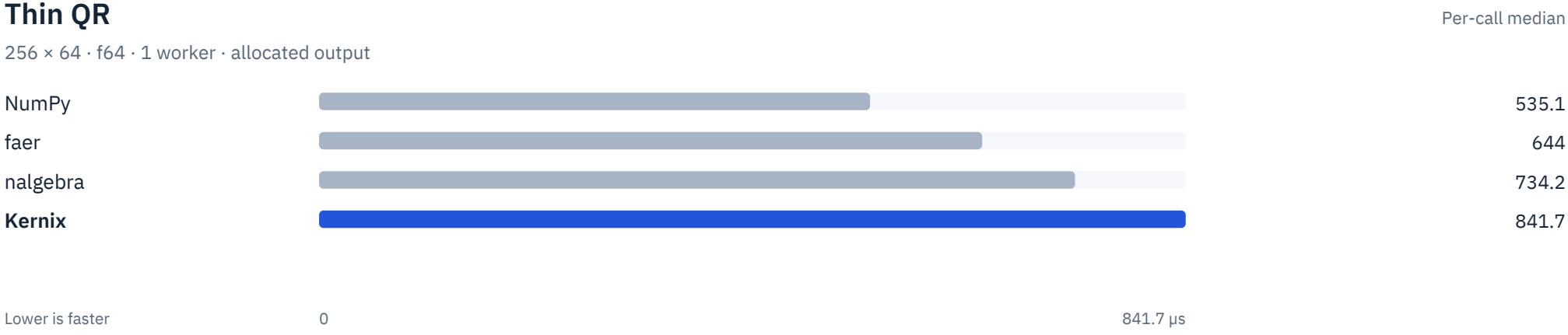
Thin QR

96 × 32 · f64 · 1 worker · allocated output



Thin QR

256 × 64 · f64 · 1 worker · allocated output



Kernix 0.10.0 / Decompositions & solvers

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · µs · NumPy BLAS: scipy-openblas

Cholesky factorization

Per-call median

64 × 64 · f64 · 1 worker · allocated output



Cholesky factorization

Per-call median

128 × 128 · f64 · 1 worker · allocated output



Kernix 0.10.0 / Decompositions & solvers

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · µs · NumPy BLAS: scipy-openblas

Cholesky factorization

256 × 256 · f64 · 1 worker · allocated output



Kernix 0.10.0 / Training & autodiff

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · µs · NumPy BLAS: scipy-openblas

Shared-variable backward

Per-call median

sum(X * X), 64 × 32; prepared graph · f64 · 1 worker · prepared graph · fresh gradients



Record + backward

Per-call median

sum(A · W), 32 × 16 · 16 × 8 · f64 · 1 worker · allocated output · self product



Kernix 0.10.0 / Training & autodiff

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · µs · NumPy BLAS: scipy-openblas

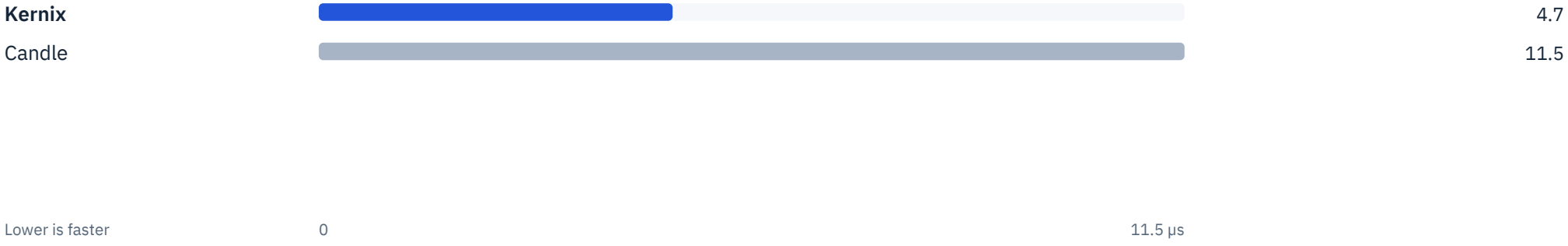
Model inference

batched normalized/sliced MLP · f64 · 1 worker · preparation as documented



Model recording + forward

batched normalized/sliced MLP · f64 · 1 worker · preparation as documented



Kernix 0.10.0 / Training & autodiff

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · µs · NumPy BLAS: scipy-openblas

Model backward

Per-call median

prepared graph → fresh gradients · f64 · 1 worker · prepared graph · fresh gradients



Parameter update

Per-call median

transactional SGD, fixed gradients · f64 · 1 worker · preparation as documented



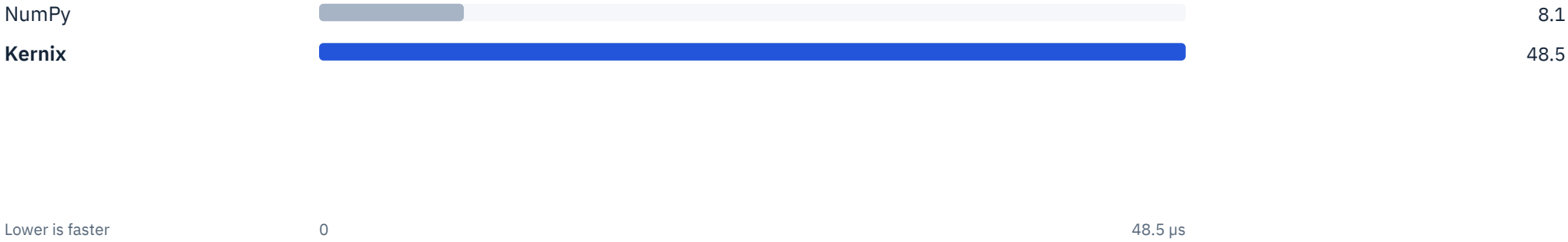
Kernix 0.10.0 / Tensor contractions

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · µs · NumPy BLAS: scipy-openblas

Coalesced tensor contraction

16 × 32 × 8 · 32 × 8 × 16 · f64 · 1 worker · allocated output

Per-call median



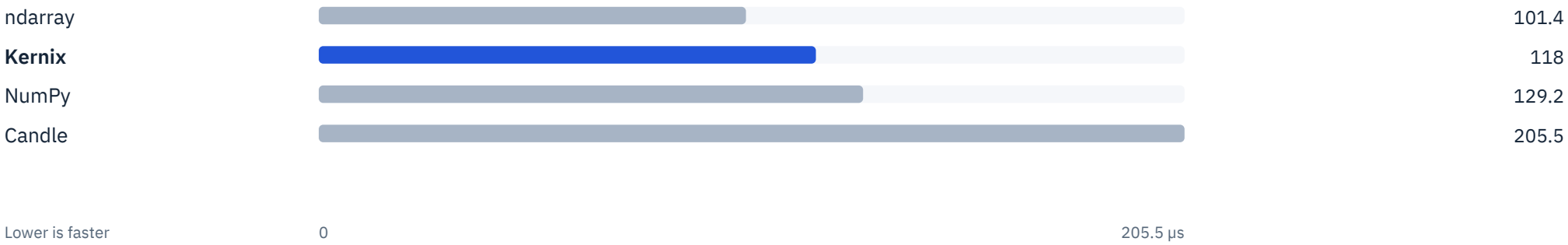
Kernix 0.10.0 / Statistics & activations

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · µs · NumPy BLAS: scipy-openblas

Row softmax

256 × 64 · f64 · 1 worker · allocated output

Per-call median



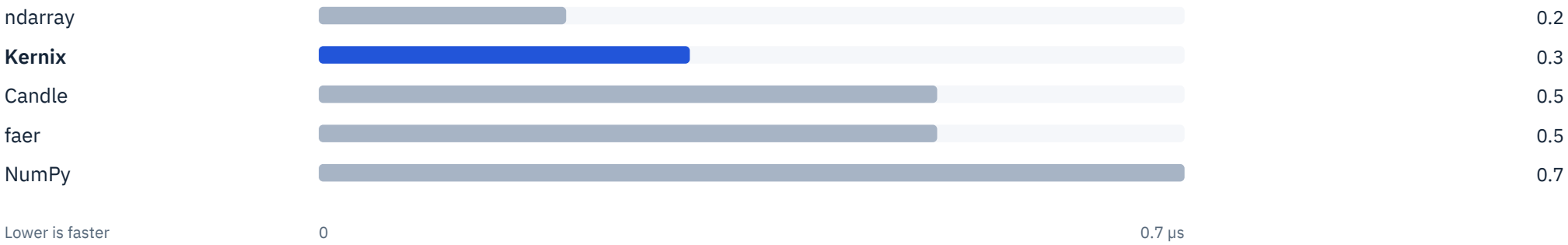
Kernix 0.10.0 / Tensor construction

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · µs · NumPy BLAS: scipy-openblas

Create zeros

Per-call median

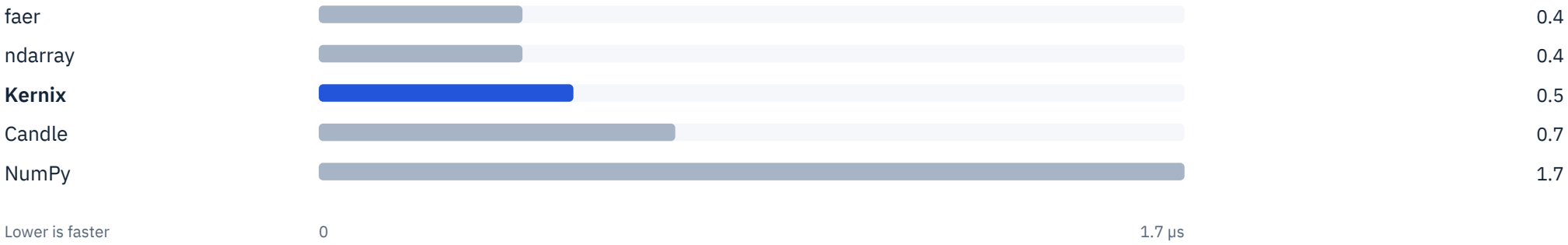
64 × 64 · f32 · 1 worker · allocated output



Create ones

Per-call median

64 × 64 · f32 · 1 worker · allocated output



Recorded precision / exact latency table

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · no rounding applied

WORKLOAD / LIBRARY	NANOSECONDS	MICROSECONDS
Matrix multiplication / faer 32 × 32 · 32 × 32 · f64	1900	1.9
Matrix multiplication / Candle 32 × 32 · 32 × 32 · f64	2700	2.7
Matrix multiplication / NumPy 32 × 32 · 32 × 32 · f64	2400	2.4
Matrix multiplication / Kernix 32 × 32 · 32 × 32 · f64	1800	1.8
Matrix multiplication / ndarray 32 × 32 · 32 × 32 · f64	1400	1.4
Matrix multiplication / nalgebra 32 × 32 · 32 × 32 · f64	1400	1.4
Matrix multiplication / ndarray 64 × 64 · 64 × 64 · f64	74700	74.7
Matrix multiplication / faer 64 × 64 · 64 × 64 · f64	9600	9.6
Matrix multiplication / Candle 64 × 64 · 64 × 64 · f64	15600	15.6
Matrix multiplication / Kernix 64 × 64 · 64 × 64 · f64	14900	14.9

Recorded precision / exact latency table

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · no rounding applied

WORKLOAD / LIBRARY	NANOSECONDS	MICROSECONDS
Matrix multiplication / NumPy 64 × 64 · 64 × 64 · f64	8200	8.2
Matrix multiplication / nalgebra 64 × 64 · 64 × 64 · f64	72100	72.1
Matrix multiplication / Kernix 128 × 128 · 128 × 128 · f64	106900	106.9
Matrix multiplication / faer 128 × 128 · 128 × 128 · f64	60600	60.6
Matrix multiplication / ndarray 128 × 128 · 128 × 128 · f64	66900	66.9
Matrix multiplication / nalgebra 128 × 128 · 128 × 128 · f64	66100	66.1
Matrix multiplication / Candle 128 × 128 · 128 × 128 · f64	178000	178
Matrix multiplication / NumPy 128 × 128 · 128 × 128 · f64	72200	72.2
Matrix multiplication / Kernix 256 × 256 · 256 × 256 · f64	827700	827.7
Matrix multiplication / NumPy 256 × 256 · 256 × 256 · f64	485700	485.7

Recorded precision / exact latency table

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · no rounding applied

WORKLOAD / LIBRARY	NANOSECONDS	MICROSECONDS
Matrix multiplication / Candle 256 × 256 · 256 × 256 · f64	833200	833.2
Matrix multiplication / faer 256 × 256 · 256 × 256 · f64	511300	511.3
Matrix multiplication / nalgebra 256 × 256 · 256 × 256 · f64	524199.99999999994	524.19999999999994
Matrix multiplication / ndarray 256 × 256 · 256 × 256 · f64	1056900	1056.9
Matrix multiplication / nalgebra 64 × 64 · 64 × 64 · f32	4800	4.8
Matrix multiplication / Candle 64 × 64 · 64 × 64 · f32	8100	8.1
Matrix multiplication / Kernix 64 × 64 · 64 × 64 · f32	7500	7.5
Matrix multiplication / faer 64 × 64 · 64 × 64 · f32	4700	4.7
Matrix multiplication / ndarray 64 × 64 · 64 × 64 · f32	35900	35.9
Matrix multiplication / NumPy 64 × 64 · 64 × 64 · f32	4700	4.7

Recorded precision / exact latency table

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · no rounding applied

WORKLOAD / LIBRARY	NANOSECONDS	MICROSECONDS
Matrix multiplication / faer 128 × 128 · 128 × 128 · f32	31800	31.8
Matrix multiplication / NumPy 128 × 128 · 128 × 128 · f32	38200	38.2
Matrix multiplication / Candle 128 × 128 · 128 × 128 · f32	98800	98.8
Matrix multiplication / ndarray 128 × 128 · 128 × 128 · f32	132000	132
Matrix multiplication / Kernix 128 × 128 · 128 × 128 · f32	54200	54.2
Matrix multiplication / nalgebra 128 × 128 · 128 × 128 · f32	31200	31.2
Matrix multiplication / faer 256 × 256 · 256 × 256 · f32	232200	232.2
Matrix multiplication / Kernix 256 × 256 · 256 × 256 · f32	403900	403.9
Matrix multiplication / Candle 256 × 256 · 256 × 256 · f32	403600	403.6
Matrix multiplication / nalgebra 256 × 256 · 256 × 256 · f32	229200	229.2

Recorded precision / exact latency table

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · no rounding applied

WORKLOAD / LIBRARY	NANOSECONDS	MICROSECONDS
Matrix multiplication / ndarray 256 × 256 · 256 × 256 · f32	224500	224.5
Matrix multiplication / NumPy 256 × 256 · 256 × 256 · f32	240900	240.9
Batched multiplication / Candle 4 × 16 × 32 · 1 × 32 × 16 · f64	4500	4.5
Batched multiplication / NumPy 4 × 16 × 32 · 1 × 32 × 16 · f64	2900	2.9
Batched multiplication / Kernix 4 × 16 × 32 · 1 × 32 × 16 · f64	2400	2.4
Matmul · 2 workers · reused output / Kernix 64 × 64 · 64 × 64 · f64	106900	106.9
Matmul · 2 workers · reused output / Kernix 128 × 128 · 128 × 128 · f64	114900	114.9
Matmul · 2 workers · reused output / Kernix 256 × 256 · 256 × 256 · f64	504099.99999999994	504.09999999999994
Matmul · 2 workers · reused output / Kernix 128 × 128 · 128 × 128 · f32	77800	77.8
Column-pivoted QR / Kernix 96 × 32 · f64	134500	134.5

Recorded precision / exact latency table

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · no rounding applied

WORKLOAD / LIBRARY	NANOSECONDS	MICROSECONDS
Column-pivoted QR / faer 96 × 32 · f64	147800	147.8
Column-pivoted QR / nalgebra 96 × 32 · f64	142400	142.4
Column-pivoted QR / Kernix 256 × 64 · f64	1042100	1042.1
Column-pivoted QR / faer 256 × 64 · f64	758700	758.7
Column-pivoted QR / nalgebra 256 × 64 · f64	1347400	1347.4
Minimum-norm least squares / Kernix 64 × 32, rank 16, 4 RHS · f64	102000	102
Minimum-norm least squares / nalgebra 64 × 32, rank 16, 4 RHS · f64	164100	164.1
Minimum-norm least squares / NumPy 64 × 32, rank 16, 4 RHS · f64	99400	99.4
Minimum-norm least squares / nalgebra 32 × 64, rank 32, 4 RHS · f64	164400	164.4
Minimum-norm least squares / Kernix 32 × 64, rank 32, 4 RHS · f64	114600	114.6

Recorded precision / exact latency table

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · no rounding applied

WORKLOAD / LIBRARY	NANOSECONDS	MICROSECONDS
Minimum-norm least squares / NumPy 32 × 64, rank 32, 4 RHS · f64	177600	177.6
Thin QR / nalgebra 96 × 32 · f64	81900	81.9
Thin QR / NumPy 96 × 32 · f64	65000	65
Thin QR / Kernix 96 × 32 · f64	131200	131.2
Thin QR / faer 96 × 32 · f64	130800.00000000001	130.80000000000001
Thin QR / faer 256 × 64 · f64	644000	644
Thin QR / Kernix 256 × 64 · f64	841700	841.7
Thin QR / NumPy 256 × 64 · f64	535100	535.1
Thin QR / nalgebra 256 × 64 · f64	734200	734.2
Cholesky factorization / NumPy 64 × 64 · f64	19300	19.3

Recorded precision / exact latency table

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · no rounding applied

WORKLOAD / LIBRARY	NANOSECONDS	MICROSECONDS
Cholesky factorization / Kernix 64 × 64 · f64	29300	29.3
Cholesky factorization / faer 64 × 64 · f64	17200	17.2
Cholesky factorization / nalgebra 64 × 64 · f64	19400	19.4
Cholesky factorization / nalgebra 128 × 128 · f64	122600	122.6
Cholesky factorization / NumPy 128 × 128 · f64	121900	121.9
Cholesky factorization / faer 128 × 128 · f64	54000	54
Cholesky factorization / Kernix 128 × 128 · f64	166600	166.6
Cholesky factorization / nalgebra 256 × 256 · f64	919200	919.2
Cholesky factorization / faer 256 × 256 · f64	235600	235.6
Cholesky factorization / Kernix 256 × 256 · f64	928100	928.1

Recorded precision / exact latency table

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · no rounding applied

WORKLOAD / LIBRARY	NANOSECONDS	MICROSECONDS
Cholesky factorization / NumPy 256 × 256 · f64	1008200	1008.2
Shared-variable backward / Candle sum(X * X), 64 × 32; prepared graph · f64	73400	73.4
Shared-variable backward / Kernix sum(X * X), 64 × 32; prepared graph · f64	10600	10.6
Record + backward / Candle sum(A · W), 32 × 16 · 16 × 8 · f64	10300	10.3
Record + backward / Kernix sum(A · W), 32 × 16 · 16 × 8 · f64	4200	4.2
Model inference / Candle batched normalized/sliced MLP · f64	8800	8.8
Model inference / Kernix batched normalized/sliced MLP · f64	3400	3.4
Model recording + forward / Candle batched normalized/sliced MLP · f64	11500	11.5
Model recording + forward / Kernix batched normalized/sliced MLP · f64	4700	4.7
Model backward / Kernix prepared graph → fresh gradients · f64	5600	5.6

Recorded precision / exact latency table

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · no rounding applied

WORKLOAD / LIBRARY	NANOSECONDS	MICROSECONDS
Model backward / Candle prepared graph → fresh gradients · f64	35300	35.3
Parameter update / Kernix transactional SGD, fixed gradients · f64	900	0.9
Coalesced tensor contraction / Kernix 16 × 32 × 8 · 32 × 8 × 16 · f64	48500	48.5
Coalesced tensor contraction / NumPy 16 × 32 × 8 · 32 × 8 × 16 · f64	8100	8.1
Row softmax / Candle 256 × 64 · f64	205500	205.5
Row softmax / ndarray 256 × 64 · f64	101400	101.4
Row softmax / Kernix 256 × 64 · f64	118000	118
Row softmax / NumPy 256 × 64 · f64	129200	129.2
Create zeros / Candle 64 × 64 · f32	500	0.5
Create zeros / faer 64 × 64 · f32	500	0.5

Recorded precision / exact latency table

INTEL(R) XEON(R) PLATINUM 8573C · Per-call median · no rounding applied

WORKLOAD / LIBRARY	NANOSECONDS	MICROSECONDS
Create zeros / Kernix 64 × 64 · f32	300	0.3
Create zeros / ndarray 64 × 64 · f32	200	0.2
Create zeros / NumPy 64 × 64 · f32	700	0.7
Create ones / Kernix 64 × 64 · f32	500	0.5
Create ones / Candle 64 × 64 · f32	700	0.7
Create ones / NumPy 64 × 64 · f32	1700	1.7
Create ones / ndarray 64 × 64 · f32	400	0.4
Create ones / faer 64 × 64 · f32	400	0.4